

Day 2 =>

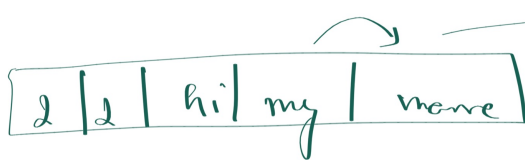
List Interface

↳ A list is an ordered collection interface in Java that allows duplicate elements

- ordered collection
- Allow duplicates
- Indexed Access
- Dynamic Sizing

Methods which was previous remain same
Some more methods are

(i) void add(^{style} int index, E element)



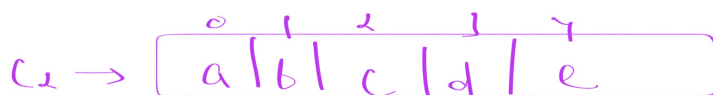
→ shifting is done in array list but not in case of linked list

```
List<String> l1 = new ArrayList<>();
```

```
l1.add(3, "my");
```

```
System.out.println(l1);
```

(2) boolean addAll(int index, Collection c)



`c1.addAll(2, c2);`

2	2	a	b	c	d	e	a	b	c
0	1	2	3	4	5	6	7	8	9

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class DAY02 {
5     public static void main(String[] args) {
6         // add(index, type of element)
7         List<String> l1 = new ArrayList<>();
8         l1.add("2");
9         l1.add("2");
10        l1.add("hi");
11        l1.add("name");
12        System.out.println(l1);
13        l1.add(index: 3, element: "my");
14        System.out.println(l1);
15
16        // addAll(index, collection) function
17        List<Integer> l2 = new ArrayList<>();
18        List<Integer> l3 = new ArrayList<>();
19        l2.add(2);
20        l2.add(3);
21        l3.add(5);
22        System.out.println(l2.addAll(index: 1, l3));
23        System.out.println(l2);
24    }
25 }
26
```

```
[2, 2, hi, name]
[2, 2, hi, my, name]
true
[2, 5, 3]
```

② E remove (int index) → returned the removed element

`c1.remove(2);`

```
// E remove(index)
System.out.println(l1);
l1.remove(index: 3);
System.out.println(l1.remove(index: 2));
System.out.println(l1);
```

```
hi
[2, 2, name]
```

④ $E \text{ set}(\text{int index}) \rightarrow$ return the particular element of the position

`c1.set(index);`

```
// E get(index)
System.out.println(l1);
System.out.println(l1.get(2));
```

```
[2, 2, name]
name
```

⑤ $E \text{ set}(\text{int index}, E \text{ element})$

C_1

0	1	2	3	4	5
7	15	8	9	10	1

`c1.set(1, 16);`

C_1

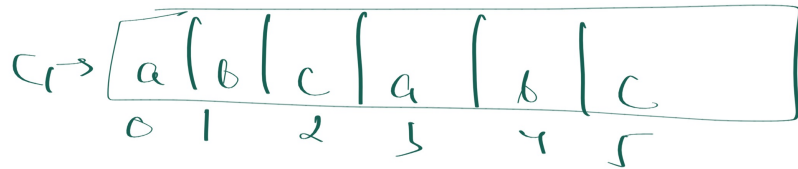
0	1	2	3	4	5
7	16	8	9	10	1

```
// E set(int index, E element)
System.out.println(l2);
l2.set(1, 8);
System.out.println(l2);
```

```
name
[2, 5, 3]
[2, 8, 3]
```

⑥ $\text{int indexof}(\text{object o}) \rightarrow$ return the index of object

`c1.indexOf(o);` $\rightarrow$ return index of o in C_1 collection of first occurrence



c1.indexOf(a) → 0

⑦ int lastIndexOf (object o)

c1.lastIndexOf(a); → 3

```

System.out.println(l2);

// int indexOf(object o)
System.out.println(l2.indexOf(8));
System.out.println(l2.indexOf(89));

// int lastIndexOf(object o)
l2.add(2);
System.out.println(l2.lastIndexOf(o: 2));
System.out.println(l2.lastIndexOf(o: 1));

```

```

1
-1
3
-1

```

- ⑧ ListIterator <E> listIterator()
- ⑨ ListIterator <E> listIterator()
- It is bidirectional
- start from any index

```

6 public class DAY02 {
7     public static void main(String[] args) {
8
9         // ListIterator<E> listIterator()
10        List<Integer> l5 = new ArrayList<>();
11        l5.add(2);
12        l5.add(9);
13        l5.add(2);
14        l5.add(9);
15        l5.add(2);
16        l5.add(9);
17        Iterator<Integer> itr1 = l5.iterator();
18        while(itr1.hasNext()){
19            System.out.print(itr1.next());
20        }
21        System.out.println();
22        ListIterator<Integer> itr2 = l5.listIterator(l5.size());
23        while(itr2.hasPrevious()){
24            System.out.print(itr2.previous());
25        }
26        // but if we want to traverse the list from the particular index then we can do that
27
28        // one time iterator work then we have to set its value again
29        System.out.println();
30        itr2 = l5.listIterator(index: 4);
31        while(itr2.hasPrevious()){
32            System.out.print(itr2.previous());
33        }
34    }
35 }

```

```

292929
929292
9292

```

Sequence →

Iterable



Collection



SequencedCollection



List

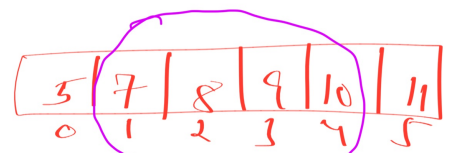
(10)

Sublist

sublist(1, 5)

included ←

→ excluded



not a new list
but just a view of
original list

→ changes in sublist or
original list will reflect in each other

```

6 public class DAY02 {
7     public static void main(String[] args) {
77         // concept of sublist(just a view of original list)
78         List<Integer> l6 = new ArrayList<>();
79         l6.add(1);
80         l6.add(2);
81         l6.add(3);
82         l6.add(4);
83         l6.add(5);
84         l6.add(6);
85         System.out.println(l6.subList(3,6));
86         // here no object is created and
87         // difference in one will show in other
88
89         // we can make object by using sublist
90         List<Integer> l7 = new ArrayList<>(l6.subList(3,6));
91         System.out.println(l7);
92     }
}

```

```

[4, 5, 6]
[4, 5, 6]
Process finished with

```

(11) List.copyOf() method

↳ copy the list which become
immutable

C₁ →

"a"	"b"	"c"	"d"
-----	-----	-----	-----

↳ list → mutable

but we want copy in C₂
such that C₂ become
immutable

```

// we want immutable list -> List.copyOf()
System.out.println(l1);
List<String> l8 = List.copyOf(l1);
System.out.println(l8);
System.out.println(l8.remove(index: 1));

```

```

[2, 2, name]
[2, 2, name]
Exception in thread "main" java.lang
at java.base/java.util.Immutable
at java.base/java.util.Immutable

```

It will return null point exception

↳ if collection or any elements are null

(12) List.of() → to create immutable list

```
public class DAY02 {  
    public static void main(String[] args) {  
        // second method is List.of()  
        List<Integer> l10 = List.of();  
        // l10.add(1);  
        // l10.add(2);  
        // System.out.println(l10);  
        // System.out.println(l10.remove(1));  
  
        // none of this will run as List.of() -> leads to empty list  
        // so 0 modification be made in it  
  
        List<Integer> l11 = List.of(1, 2, 3, 4, 5);  
        System.out.println(l11);  
        // System.out.println(l11.remove(3)); -> will give exception  
    }  
}
```

max elements 10

↓
overloaded method